

Full Stack Developer — Step 7

Deployment & Job Preparation

Roadmap objective: Deploy a full-stack application and prepare evidence for hiring

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Build pipeline
- Environment variables
- Frontend hosting
- Backend hosting
- Database configuration
- CORS
- Logs
- Custom domains
- Resume
- Portfolio
- Applications

Core concepts

1. A deployed full-stack app has at least three concerns: frontend delivery, backend/API availability and persistent database connectivity.
2. Environment-specific URLs and secrets should be configuration, not hard-coded source.
3. After deployment, test the same user flows that worked locally: login, API requests, database writes, file/resource loading and error states.
4. For job applications, present projects with live link, repository, technologies and a concise explanation of your contribution.

Concept map

```
Deployment & Job Preparation
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
```

+--> Document the decision

Detailed study guide

1. Build pipeline

Study build pipeline through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Environment variables

Study environment variables through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Frontend hosting

Study frontend hosting through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Backend hosting

Study backend hosting through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Database configuration

Study database configuration through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. CORS

Study cors through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Deployment checklist

Frontend
-> correct API base URL
-> static assets load

Backend

- > PORT configured
- > CORS configured
- > environment variables set
- > health endpoint works

Database

- > production credentials
- > migrations/schema applied
- > connectivity verified

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Render	App/backend hosting	Useful for services
Vercel	Frontend/serverless platform	Strong frontend workflow
GitHub	Code collaboration	Repository/CI integration

Common mistakes

- Hard-coded localhost URLs
- Missing production environment variables
- Wrong CORS origin
- Not checking logs
- Assuming local success means production success

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Write a deployment architecture diagram.
- Write down what you expected, what happened, and why.

Lab 2

- List every environment variable.
- Write down what you expected, what happened, and why.

Lab 3

- Verify frontend API URLs.
- Write down what you expected, what happened, and why.

Lab 4

- Verify backend CORS.
- Write down what you expected, what happened, and why.

Lab 5

- Verify database connectivity.
- Write down what you expected, what happened, and why.

Lab 6

- Test login/read/write/error flows after deployment.
- Write down what you expected, what happened, and why.

Mini-project

Deploy a full-stack project with frontend, backend and database. Create a deployment checklist, document environment variables, test production flows and prepare a polished project README.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What changes between local and production?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you manage secrets?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is CORS in deployment?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you debug production?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you verify DB connectivity?

- Answer structure: definition → example → trade-off/limitation → practical use.

What makes a project job-ready?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Hosting	Write this in your own words.	Give one project example.
Environment	Write this in your own words.	Give one project example.
CORS	Write this in your own words.	Give one project example.
DNS	Write this in your own words.	Give one project example.
Build	Write this in your own words.	Give one project example.
Runtime	Write this in your own words.	Give one project example.
Log	Write this in your own words.	Give one project example.
Health check	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Secret	Write this in your own words.	Give one project example.
Rollback	Write this in your own words.	Give one project example.

Self-test

- Explain Build pipeline without using its textbook definition.
- Show a small example of Environment variables.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Build pipeline.
- I can explain and demonstrate Environment variables.
- I can explain and demonstrate Frontend hosting.
- I can explain and demonstrate Backend hosting.
- I can explain and demonstrate Database configuration.
- I can explain and demonstrate CORS.
- I can explain and demonstrate Logs.
- I can explain and demonstrate Custom domains.
- I can explain and demonstrate Resume.
- I can explain and demonstrate Portfolio.
- I can explain and demonstrate Applications.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://render.com/docs>
- <https://vercel.com/docs>
- <https://docs.github.com/>
- <https://www.linkedin.com/help/linkedin>

Recommended videos

- Full Stack MERN App Deployment on Render (Existing DB resource 195)

- [How to Write a Software Engineer Resume - freeCodeCamp](#) (Existing DB resource 114)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.